gmx No.14 funding_fee

https://github.com/xiaoyu1998/

Resolution:2560×1440(2K)
https://github.com/gmx-io/gmx-synthetics/tree/v2.1

Entries

executeOrder
contracts/order/ExecuteOrderUtils.sol

```
PositionUtils.updateFundingAndBorrowingState(
 eventData = processOrder(
```

processOrder
contracts/order/ExecuteOrderUtils.sol

```
return IncreaseOrderUtils.processOrder(params);
```

processOrder
contracts/order/IncreaseOrderUtils.sol

```
IncreasePositionUtils.increasePosition(
```

updateFundingAndBorrowingState
contracts/position/PositionUtils.sol

```
MarketUtils.updateFundingState(
MarketUtils.updateCumulativeBorrowingFactor(
```

updateCumulativeBorrowingFactor
contracts/market/MarketUtils.sol

```
uint256 delta) = getNextCumulativeBorrowingFactor(
incrementCumulativeBorrowingFactor(
```

getNextCumulativeBorrowingFactor
contracts/market/MarketUtils.sol

```
uint256 borrowingFactorPerSecond = getBorrowingFactorPerSecond(
uint256 nextCumulativeBorrowingFactor =
cumulativeBorrowingFactor + delta;
```

getBorrowingFactorPerSecond
contracts/market/MarketUtils.sol

```
getReservedUsd(
getPoolUsdWithoutPnl(dataStore, market, prices, isLong, false);
uint256 reservedUsdAfterExponent =
Precision.applyExponentFactor(reservedUsd,
borrowingExponentFactor);
uint256 reservedUsdToPoolFactor =                    借出比例
Precision.toFactor(reservedUsdAfterExponent, poolUsd);
uint256 borrowingFactor = getBorrowingFactor(dataStore,
market.marketToken, isLong);
return Precision.applyFactor(reservedUsdToPoolFactor,
borrowingFactor);
```

Market

updateFundingState
contracts/market/MarketUtils.sol

```
getNextFundingAmountPerSize(dataStore, market, prices);
applyDeltaToFundingFeeAmountPerSize(
applyDeltaToClaimableFundingAmountPerSize(
```

getNextFundingAmountPerSize
contracts/market/MarketUtils.sol

```
cache.longOpenInterest = cache.openInterest.long.longToken
+ cache.openInterest.long.shortToken;
cache.shortOpenInterest = cache.openInterest.short.longToken
+ cache.openInterest.short.shortToken;
cache.fundingUsd =                                   总资金费
Precision.applyFactor(cache.sizeOfLargerSide,
cache.durationInSeconds * result.fundingFactorPerSecond);
cache.fundingUsdForLongCollateral =                  eth抵押需要支付的资金费
Precision.mulDiv(cache.fundingUsd,
cache.openInterest.long.longToken, cache.longOpenInterest);
cache.fundingUsdForShortCollateral =                 usdt抵押需要支付的资金费
Precision.mulDiv(cache.fundingUsd,
cache.openInterest.long.shortToken, cache.longOpenInterest);
result.fundingFeeAmountPerSizeDelta.long.longToken =
getFundingAmountPerSizeDelta(                        每usd,eth抵押需要承担的资金费
result.fundingFeeAmountPerSizeDelta.long.shortToken =
getFundingAmountPerSizeDelta(                        每usd,usdt抵押需要承担的资金费
result.claimableFundingAmountPerSizeDelta.short.longToken =
getFundingAmountPerSizeDelta(                        每usd,可获得的eth资金费
result.claimableFundingAmountPerSizeDelta.short.shortToken =
getFundingAmountPerSizeDelta(                        每usd,可获得的usdt资金费
```

Position

increasePosition
contracts/position/IncreasePositionUtils.sol

```
(cache.priceImpactUsd, cache.priceImpactAmount, cache.sizeDeltaInTokens,
cache.executionPrice) = PositionUtils.getExecutionPriceForIncrease(params,
prices.indexTokenPrice);
(cache.collateralDeltaAmount, fees) = processCollateral(
```

increasePosition
contracts/position/IncreasePositionUtils.sol

```
PositionPricingUtils.PositionFees memory fees =
PositionPricingUtils.getPositionFees(getPositionFeesParams);
collateralDeltaAmount -= fees.totalCostAmount.toInt256();
```

getPositionFees
contracts/pricing/PositionPricingUtils.sol

```
uint256 borrowingFeeUsd =
MarketUtils.getBorrowingFees(params.dataStore, params.position);
fees.funding.latestFundingFeeAmountPerSize =
MarketUtils.getFundingFeeAmountPerSize(              需要支付
fees.funding.latestLongTokenClaimableFundingAmountPerSize =
MarketUtils.getClaimableFundingAmountPerSize(  可以获得的Eth
fees.funding.latestShortTokenClaimableFundingAmountPerSize =
MarketUtils.getClaimableFundingAmountPerSize(  可以获得的Usdt
```

Pricing

假设ETH/USDT市场状态：
多头未平仓量：$60,000
空头未平仓量：$40,000
用户开多仓：$10,000
价格影响指数：2
价格影响因子：0.001

计算过程：
initialDiffUsd = |60,000 - 40,000| = $20,000
nextLongOpenInterest = 60,000 + 10,000 = $70,000
nextShortOpenInterest = $40,000
nextDiffUsd = |70,000 - 40,000| = $30,000

hasPositiveImpact = 30,000 < 20,000 = false
// 应用影响因子
initialImpact = 20,000 ^ 2 * 0.001 = 400,000,000 * 0.001 = 400,000
nextImpact = 30,000 ^ 2 * 0.001 = 900,000,000 * 0.001 = 900,000

deltaDiffUsd = 400,000 - 900,000 = -500,000
priceImpactUsd = -500,000 // 负值，用户需要支付成本
priceImpactAmount = priceImpactUsd / indexTokenPrice.max;
sizeDeltaInTokens = baseSizeDeltaInTokens + priceImpactAmount;

getExecutionPriceForIncrease
contracts/position/PositionUtils.sol

```
int256 priceImpactUsd = PositionPricingUtils.getPriceImpactUsd(
priceImpactAmount = priceImpactUsd / indexTokenPrice.max.toInt256();
sizeDeltaInTokens = baseSizeDeltaInTokens.toInt256() +
priceImpactAmount;
```

getPriceImpactUsd
contracts/pricing/PositionPricingUtils.sol

```
int256 priceImpactUsd = _getPriceImpactUsd(params.dataStore,
params.market.marketToken, openInterestParams);
```

_getPriceImpactUsd
contracts/pricing/PositionPricingUtils.sol

```
return PricingUtils.getPriceImpactUsdForSameSideRebalance(
```

getPriceImpactUsdForSameSideRebalance
contracts/pricing/PricingUtils.sol

```
uint256 deltaDiffUsd = Calc.diff(
    applyImpactFactor(initialDiffUsd, impactFactor, impactExponentFactor),
    applyImpactFactor(nextDiffUsd, impactFactor, impactExponentFactor)
int256 priceImpactUsd = Calc.toSigned(deltaDiffUsd, hasPositiveImpact);
```